Week 7

Embedded ROS 2

Jaerock Kwon, PhD
Associate Professor
Electrical and Computer Engineering
University of Michigan-Dearborn

Why a microcontroller at all

The laptop runs Linux. Linux is **not** a real-time operating system.

It cannot promise to close a position loop at 200 Hz.
It can **usually** do it — which is worse than a clear no,
because the failures are rare and load-dependent.

Split by timing guarantee

Runs on the	Runs on the
Steering position loop	SLAM
Safety state machine	Planning
PWM generation	Perception, logging
deterministic	**best effort**

ADR D3 adopted a **single** Teensy, replacing Pixhawk 6C + Arduino Nano.

A single controller is a single point of failure

And the ADR says so. D3 was adopted **conditional** on:

> *SBUS into the Teensy for normal closed-loop override, **plus** a hardware RC signal MUX on the E-stop chain as the independent deeper fallback.*
>
> ***The condition is not optional; it is what makes the single-point-of-failure objection answerable.***

Note what the record does **not** do: claim the objection was wrong.

It names the objection and **buys it off with a mechanism.**

And a pre-registered fallback

E4: if the Teensy loop cannot hold ≤ 1° steady-state with no sustained oscillation at bring-up Stage 1 —

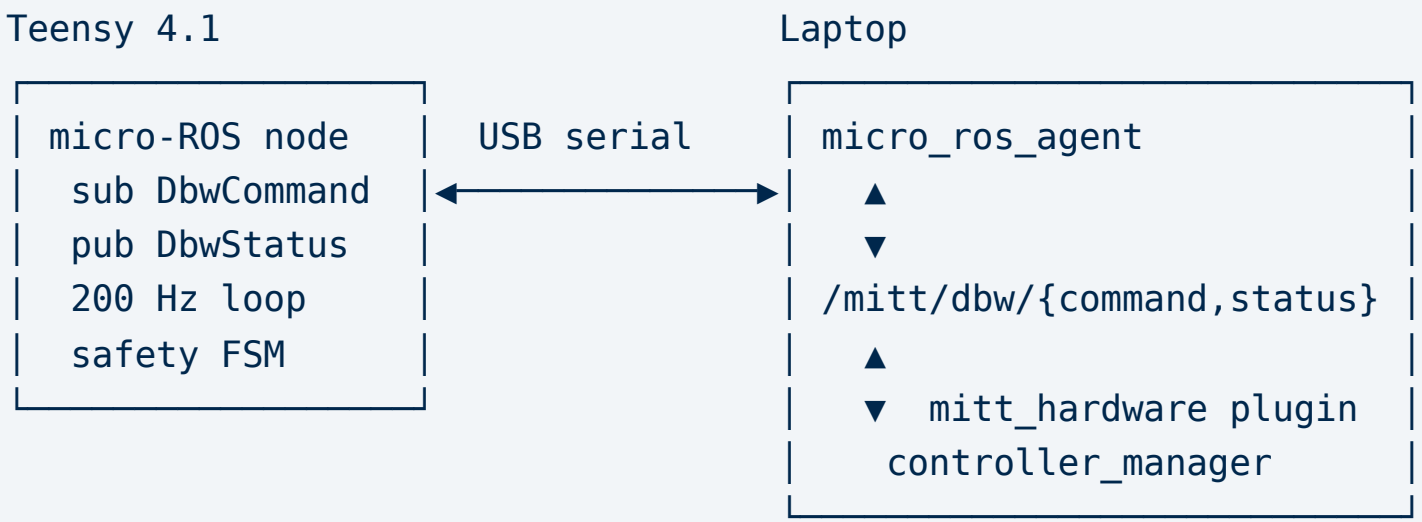
adopt the motion-controller daughterboard
rather than tuning without bound.

The trigger was written **before** the attempt.

That is the difference between a plan and a hope.

micro-ROS

A real ROS 2 node on the microcontroller



One transport, one clock, typed messages.

ADR-SW1 — so a single `ros2 topic echo` shows **both sides of the control loop.**

That is not a convenience

Debugging a control loop whose command and feedback arrive over **different transports on different clocks** is a category of pain this design simply refuses to enter.

Practical note: `micro_ros_agent` is **not** in the Humble apt repos.

Build it from source **now**, during track work — not in Week 13 when it blocks the vehicle.

Record the commit you built.

The interface contract

`DbwCommand` — laptop → Teensy, ≥ 50 Hz

Field	
`steering_angle`	**radians**, clamped in firmware to ±0.3927 (±22.5°)
`speed`	signed m/s, positive forward

Two fields. That is the whole command.

`DbwStatus` — Teensy → laptop, ≥ 50 Hz

Field	
`steering_angle`	**measured**, from the load-side sensor
`steering_setpoint`	the setpoint being tracked, **echoed back**
`wheel_speed`, `drive_ticks`	**"PPR is a MEASURED constant… do NOT assume 52"**
`mode`	ESTOP / MANUAL_RC / AUTONOMOUS
`faults`	a **bitfield**

> **Authoritative — never derived from the motor encoder, which sits upstream of the gearbox backlash.**

Three choices worth stealing

Echo the setpoint back. The firmware already knows it — but now commanded and measured sit on **one time axis in one bag.** **"Did it not follow, or did it not receive?"** becomes answerable from a single recording.

Radians in the message, degrees never. One unit crosses the boundary. Unit confusion at an interface is a classic, expensive bug.

A bitfield, not a boolean. `is_faulted: true` tells you to stop. Six named causes tell you **why** — which is what you need at 2 a.m. with the vehicle on blocks.

The timing contract

`DbwCommand` / `DbwStatus` rate	≥ 50 Hz
Steering position loop	≥ 200 Hz on the Teensy
Command staleness → ESTOP	> 500 ms
Joystick → wheel latency	≤ 100 ms at p95
USB dropouts	**zero** over ≥ 30 min

These are acceptance gates, not aspirations.

`FAULT_SETPOINT_STALE` exists to make the 500 ms rule **observable** rather than merely intended.

`hardware_interface`

The other side of the boundary

```
return_type read(const rclcpp::Time&, const rclcpp::Duration&) override;
// hardware state -> state_interfaces

return_type write(const rclcpp::Time&, const rclcpp::Duration&) override;
// command_interfaces -> hardware
```

For MRider:

- `read()` — take the latest `DbwStatus`, fill the joint state interfaces
- `write()` — take the controller's commands, publish a `DbwCommand`

There was never working code here to reuse

B-MROVER's `carlikebot_system.cpp` is an **unmodified upstream demo stub**:

- `read()` **echoes the command back as state**
- `write()` only logs

(finding F3)

A stub that echoes commands back as state is the worst possible thing to inherit. In simulation, and in any test that does not touch hardware, it looks like it works perfectly.

`mitt_hardware` is new work

It is the **Software track's keystone**.

Merge 3, Week 13 exists to prove ADR-SW4 with it:

> *The identical teleop launch runs in sim and on hardware, differing **only** in this plugin.*

Track work this week

Track	
Electrical	Bench kit up. Blink, then read the angle sensor open-loop — Stage 0
Chassis	Teardown; re-measure M1/M2/M3; start the mounting design
Simulation	First functional test — make `colcon test` assert something that is not a linter
Software	`mitt_hardware` skeleton: load it, activate it, publish nothing yet

Charters were due today. If yours contains an undated dependency,
fix that before you write any more code.

Next week

★ **Mid-semester presentation** — 12 min + 5 min questions, everyone speaks.

Plus the safety lecture: the authority ladder, the failsafe matrix,
and the staged bring-up protocol.

From Week 9 the hardware tracks start energising motors.
Nothing after next week is wheels-off by accident.